

附件(解題詳細過程以及心得)

B1229062_林冠妤

在這份檔案我會講一下我的題目思路，包含一開始的發想，到最後答案產生的過程；再來，我會講一下我在這學期的軟硬體協同設計課程的心得，包含我作業、課程總心得，最後我要說一下這門課的建議(也不算真的建議，是一開始對於這堂課的期許)。

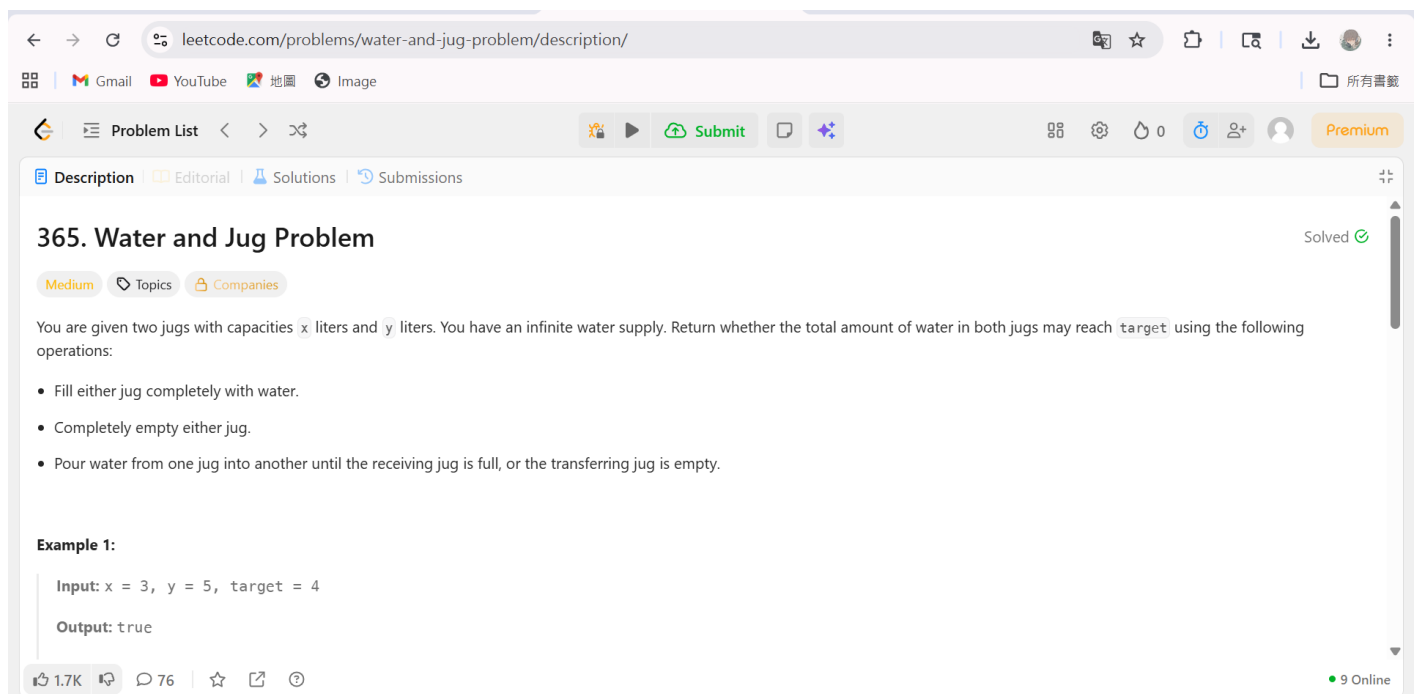
一、這一題Q3:

Water Jug Problem

- Given two jugs with capacities of 3 liters and 5 liters, the goal is to measure exactly 4 liters of water.

Find a state diagram to represent the different possible states of the jugs and the transitions between them.

1. **LeetCode:** 我一開始寫這個題目的時候是先去看LeetCode，因為有看過這個題目，然後我查詢之後真的，在365題，有一樣的題目，只是他不是要我畫state diagram，他是要我寫code去解他的問題，然後我複習一下之前的C++，有再次解出來。(其實只是想要回顧一下記憶)

The image is a screenshot of a web browser displaying the LeetCode problem page for '365. Water and Jug Problem'. The browser's address bar shows the URL 'leetcode.com/problems/water-and-jug-problem/description/'. The page header includes navigation links like 'Problem List', 'Submit', and 'Premium'. The problem title '365. Water and Jug Problem' is prominently displayed, along with a 'Solved' status indicator. Below the title, the difficulty is marked as 'Medium'. The problem description states: 'You are given two jugs with capacities x liters and y liters. You have an infinite water supply. Return whether the total amount of water in both jugs may reach target using the following operations:'. Three operations are listed: 1. Fill either jug completely with water. 2. Completely empty either jug. 3. Pour water from one jug into another until the receiving jug is full, or the transferring jug is empty. An 'Example 1:' is provided with 'Input: x = 3, y = 5, target = 4' and 'Output: true'. At the bottom, there are statistics: '1.7K' likes, '76' comments, and '9 Online' users.

(上圖是LeetCode題目)

● 他有放完整的example:

Example 1:

Input: x = 3, y = 5, target = 4

Output: true

Explanation:

Follow these steps to reach a total of 4 liters:

1. Fill the 5-liter jug (0, 5).
2. Pour from the 5-liter jug into the 3-liter jug, leaving 2 liters (3, 2).
3. Empty the 3-liter jug (0, 2).
4. Transfer the 2 liters from the 5-liter jug to the 3-liter jug (2, 0).
5. Fill the 5-liter jug again (2, 5).
6. Pour from the 5-liter jug into the 3-liter jug until the 3-liter jug is full. This leaves 4 liters in the 5-liter jug (3, 4).
7. Empty the 3-liter jug. Now, you have exactly 4 liters in the 5-liter jug (0, 4).

Reference: The [Die Hard](#) example.

Example 2:

Input: x = 2, y = 6, target = 5

Output: false

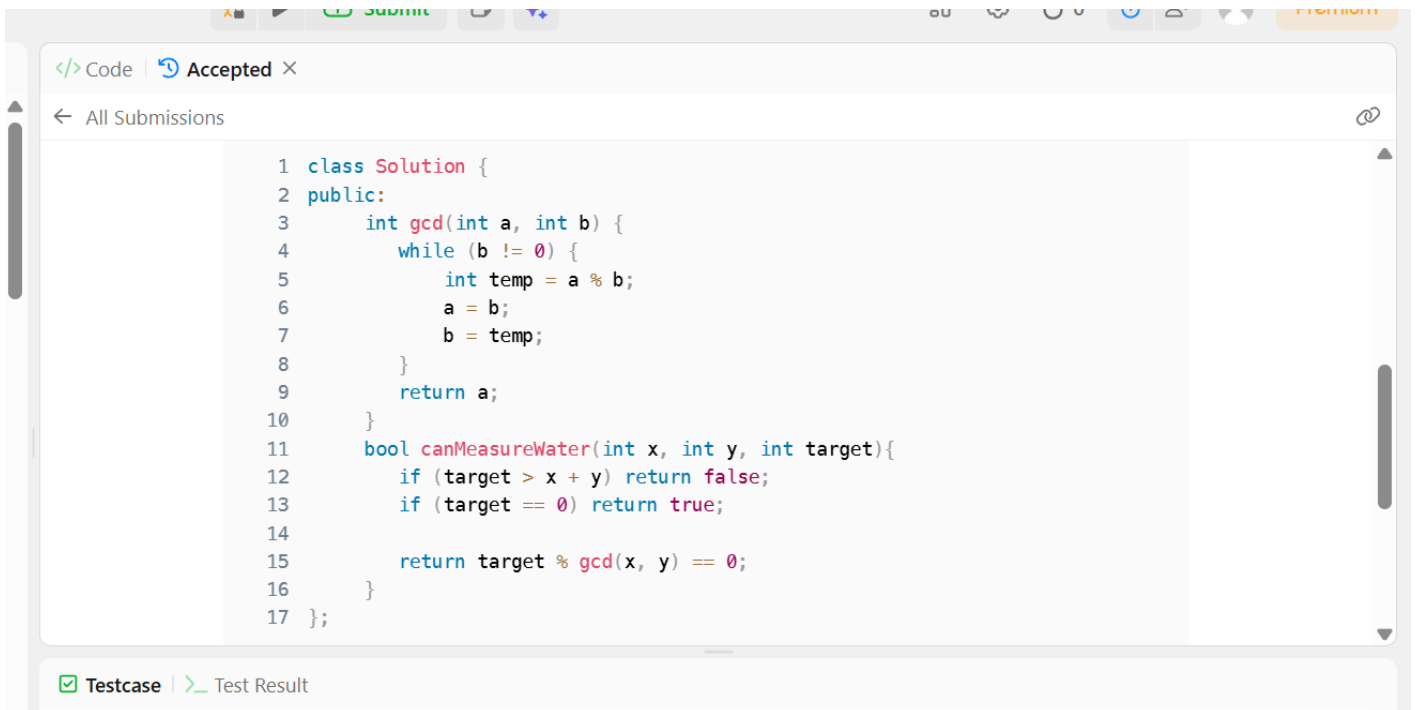
Example 3:

Input: x = 1, y = 2, target = 3

Output: true

Explanation: Fill both jugs. The total amount of water in both jugs is equal to 3 now.

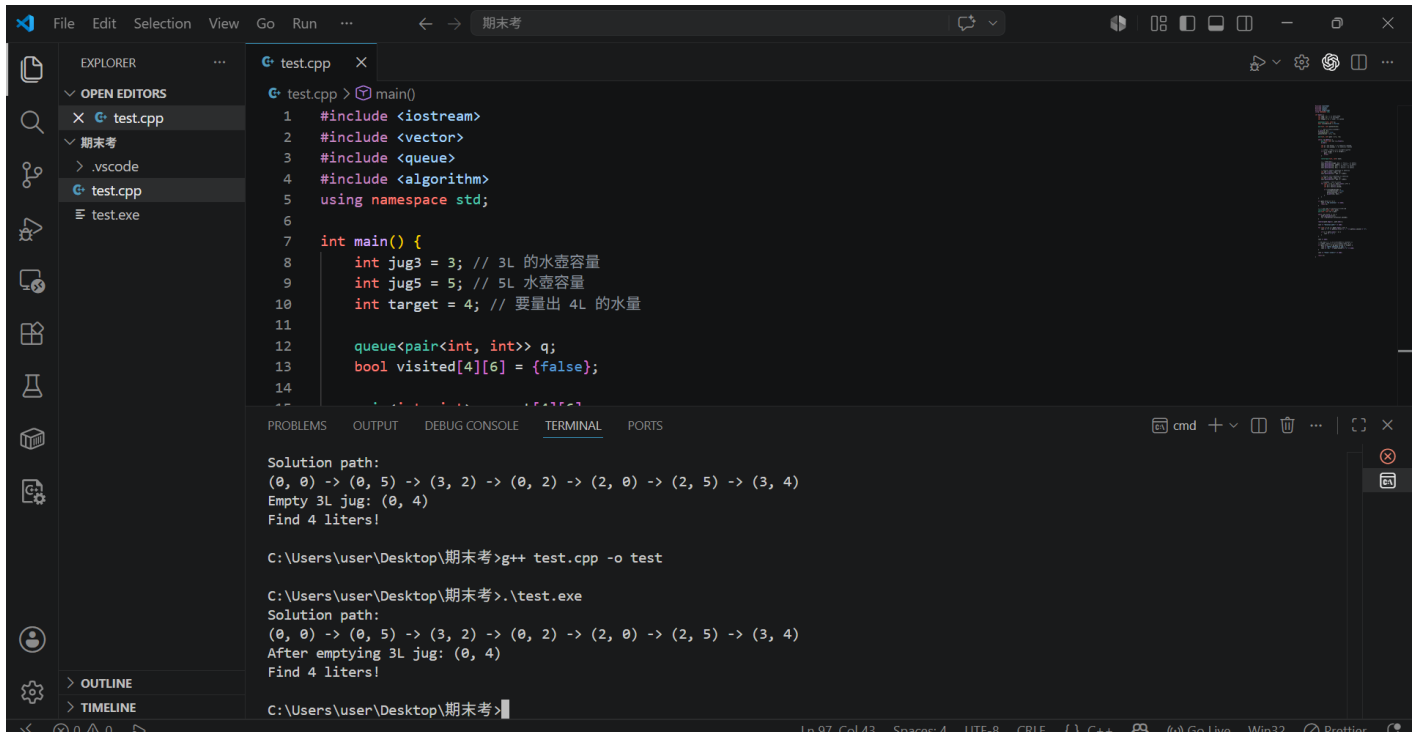
● 然後我也有重新打一次C++ code:



```
1 class Solution {
2 public:
3     int gcd(int a, int b) {
4         while (b != 0) {
5             int temp = a % b;
6             a = b;
7             b = temp;
8         }
9         return a;
10    }
11    bool canMeasureWater(int x, int y, int target){
12        if (target > x + y) return false;
13        if (target == 0) return true;
14
15        return target % gcd(x, y) == 0;
16    }
17 };

Testcase | Test Result
```

2. BFS Tree:再來就是我也有想要資料結構演算法的BFS Tree，然後也有用BFS的code複習一次(是在本機端的 VS code做的)，也有印出老師要的題目diagram結果



The screenshot shows the VS Code interface with a C++ file named `test.cpp` open. The code implements a BFS solution for the 3-jug problem. The terminal output shows the solution path: `(0, 0) -> (0, 5) -> (3, 2) -> (0, 2) -> (2, 0) -> (2, 5) -> (3, 4)`. It also shows the command `g++ test.cpp -o test` and the execution of `test.exe`.

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;

int main() {
    int jug3 = 3; // 3L 的水壺容量
    int jug5 = 5; // 5L 水壺容量
    int target = 4; // 要量出 4L 的水量

    queue<pair<int, int>> q;
    bool visited[4][6] = {false};

    // ... (rest of the code) ...

    Solution path:
    (0, 0) -> (0, 5) -> (3, 2) -> (0, 2) -> (2, 0) -> (2, 5) -> (3, 4)
    Empty 3L jug: (0, 4)
    Find 4 liters!

    C:\Users\user\Desktop\期末考>g++ test.cpp -o test

    C:\Users\user\Desktop\期末考>.\test.exe
    Solution path:
    (0, 0) -> (0, 5) -> (3, 2) -> (0, 2) -> (2, 0) -> (2, 5) -> (3, 4)
    After emptying 3L jug: (0, 4)
    Find 4 liters!

    C:\Users\user\Desktop\期末考>
```

- 以下是程式碼(含註解):

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;

int main() {
    int jug3 = 3; // 3L 的水壺容量
    int jug5 = 5; // 5L 水壺容量
    int target = 4; // 要量出 4L 的水量

    queue<pair<int, int>> q;
    bool visited[4][6] = {false};

    pair<int, int> parent[4][6];

    // 從一開始兩個水壺都沒水開始
    q.push({0, 0});
    visited[0][0] = true;
    parent[0][0] = {-1, -1};

    pair<int, int> goal = {-1, -1};

    while (!q.empty()) {
```

```

pair<int, int> cur = q.front();
q.pop();

int a = cur.first; // 3L 的水壺目前的水量
int b = cur.second; // 5L 的水壺目前的水量

// 只要有一個水壺剛好是 4L，就算找到答案
if (a == target || b == target) {
    goal = cur;
    break;
}

vector<pair<int, int>> next;

// 所有可能動作：
next.push_back({jug3, b}); // F3 = 裝滿 3L 的水壺
next.push_back({a, jug5}); // F5 = 裝滿 5L 的水壺
next.push_back({0, b}); // E3 = 清空 3L 的水壺
next.push_back({a, 0}); // E5 = 清空 5L 的水壺

// P35 = 3L 的水壺中的水倒到 5L 的水壺裡
int p35 = min(a, jug5 - b);
next.push_back({a - p35, b + p35});

// P53 = 5L 水壺中的水倒到 3L 的水壺裡
int p53 = min(b, jug3 - a);
next.push_back({a + p53, b - p53});

// 沒走過的狀態才放進 queue
for (int i = 0; i < next.size(); i++) {
    int na = next[i].first;
    int nb = next[i].second;

    if (!visited[na][nb]) {
        visited[na][nb] = true;
        parent[na][nb] = cur;
        q.push({na, nb});
    }
}

if (goal.first == -1) {
    cout << "No solution." << endl;
    return 0;
}

```

```

}

// 從 goal 往前追 parent，找回完整路徑
vector<pair<int, int>> path;
pair<int, int> cur = goal;

while (cur.first != -1) {
    path.push_back(cur);
    cur = parent[cur.first][cur.second];
}

reverse(path.begin(), path.end());

cout << "Solution path:" << endl;

for (int i = 0; i < path.size(); i++) {
    cout << "(" << path[i].first << ", " << path[i].second << ")";

    if (i != path.size() - 1) {
        cout << " -> ";
    }
}

cout << endl;

// 如果最後是像 (x, 4)，x>0，代表 5L 裡已經有 xL
// 這時候也可以把 xL 倒掉，變成更乾淨的 (0, 4)
if (goal.first != 0 && goal.second == target) {
    cout << "After emptying 3L jug: ";
    cout << "(0, " << goal.second << ")" << endl;
}

cout << "Find 4 liters!" << endl;

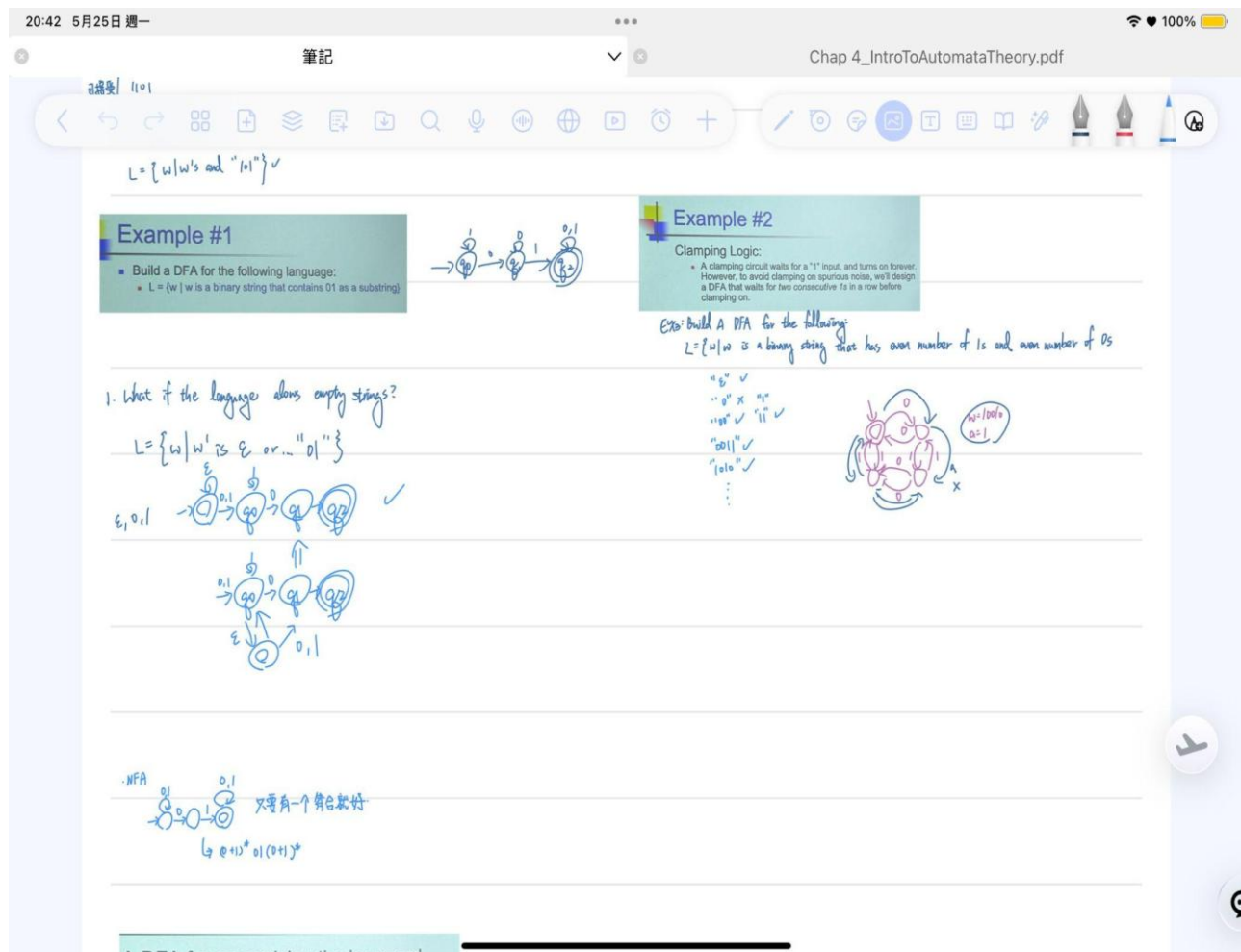
return 0;
}

```

- 上面我把聯想到的都複習完後，結果有出來了，現在就是真正來回答老師的問題。

3. 期末考題的Q3回答:

- 在這邊我想到要畫的圖就是想到這學期教的DFA，那在這邊的話我一樣先複習一下老師上課的ppt，以及看我自己的上課筆記



- 複習完了之後，我就開始審題，把有疑問的都問 ai (ChatGPT)，我問的問題有：

(1)Q：我想要問一下這邊我要將 Water Jug Problem 的問題用 DFA State Diagram 畫出來的話，我是要將所有可能的狀態都要話出來嗎？我要找出結果是 4L 的狀況(這邊也想要問一下我是要在兩個水壺中出現一個 4L 就結束嗎？還是要像LeetCode 365的範例(你可以查一下LeetCode)一樣還要將另外一個水壺清空？

A:(因為 ChatGPT 回我一長串，我這邊只放她最後給的答案)

6. 最後簡單結論

你可以這樣記：

問題	建議答案
DFA State Diagram 要畫全部嗎？	正式來說要畫所有 reachable states，不只答案路徑
找到 4L 是不是一個水壺有 4L 就可以？	可以，通常一個水壺剛好 4L 就算成功
一定要把另一個水壺清空嗎？	不一定
但作業畫圖最穩的終點？	建議畫到 (0, 4)，最清楚、最像標準範例
LeetCode 365 怎麼判定？	常見條件是 $x = target$ 、 $y = target$ 、或 $x + y = target$ 任一成立即可

(2)Q: 所以你的意思 我的diagram 就是要畫出當前的 state diagram 後面會到的所有 state 對嗎? 這樣的話也包含self loop吧(因為上課的時候都有畫到)

但是這邊有一個小重點

如果是 演算法搜尋圖 / BFS Tree, 通常不畫 self-loop, 因為它沒有幫助找到答案。

可是如果你是畫 DFA State Diagram, 尤其老師上課 DFA 有畫 self-loop, 那就應該畫, 因為 DFA 強調:

每一個 state 在每一個 input / operation 下, 都要有明確的下一個 state。

所以你可以這樣分:

圖的類型	要不要畫所有後繼 state	要不要畫 self-loop
BFS Tree / Search Tree	不一定, 只畫搜尋過程	通常不用
Solution Path	不用, 只畫答案路徑	不用
State Diagram	要畫 reachable states 的轉移	可以畫
DFA State Diagram	最正式要畫完整轉移	建議畫 self-loop

你作業最穩的做法

如果老師說要畫 DFA State Diagram, 你可以畫:

1. 所有從 (0, 0) 可以到達的 reachable states
2. 每個 state 的合法操作轉移
3. 會回到自己的操作畫成 self-loop
4. 把目標 state, 例如 (0, 4), 畫成 double circle / accepting state

A:

(3)Q: 那這邊我還想要問一下, 因為上課的時候有教過dfa、nfa、e-nfa, 我還要畫後兩個的圖嗎?

4. 你可以交哪些圖比較穩?

我會建議你交這幾張就好:

圖	要不要交	原因
DFA State Diagram	要	最符合老師教的 state diagram / DFA 概念
BFS Tree	可以交	可以展示你怎麼搜尋到 4L
Solution Path	可以交	讓老師一眼看到答案路徑
NFA	不用	這題的操作結果是確定的
ϵ -NFA	不用	這題沒有不經操作就轉移的狀態

我會建議你: 不用主動畫 NFA 和 ϵ -NFA, 除非老師題目明確要求「三種都要畫」或「比較 DFA / NFA / ϵ -NFA」。

以你的 Water Jug Problem 來說, 最適合交的是:

DFA State Diagram / State Diagram + BFS Tree / solution path

因為水壺問題本質上是「每個狀態經過某個操作, 會轉移到下一個狀態」, 很適合用 state diagram 表示。

A:

(4)Q: (因為ai 和我說了一下圖適合用的部分, 所以我還是把題目附上去, 但沒有要求他給我考試答案)

對, 看到題目後, 我會更確定地建議你:

這題主要要畫「Water Jug Problem 的 State Diagram」, 不一定要硬畫成正式 DFA, 也不用畫 NFA、 ϵ -NFA。

題目原文是:

Find a state diagram to represent the different possible states of the jugs and the transitions between them.

所以老師要看的重點應該是:

1. 不同可能狀態: 例如 (0, 0), (3, 0), (0, 5), (3, 5), (0, 2), (3, 2), (2, 0), (2, 5), (3, 4), (0, 4) 等
2. 狀態之間怎麼轉移: 裝滿、倒空、互倒
3. 最後如何到達 4L: 例如到達 (0, 4) 或 (3, 4)

A:

(5)Q: 我想要問一下我的圖設定可以怎麼寫，就是到下一個state過程的縮寫

建議使用的縮寫

你可以這樣設定：

縮寫	完整意思	中文意思
F3	Fill 3L jug	裝滿 3L 水壺
F5	Fill 5L jug	裝滿 5L 水壺
E3	Empty 3L jug	倒空 3L 水壺
E5	Empty 5L jug	倒空 5L 水壺
P35	Pour 3L jug into 5L jug	3L 倒入 5L
P53	Pour 5L jug into 3L jug	5L 倒入 3L

A:

- 我以上的對答完整記錄在：<https://chatgpt.com/share/6a1468bc-74fc-83a9-a70f-888fdb6537d8>

- 再來就是自己手搓答案的部分了，那畫圖前我要先把設定設好，不然圖會很擠，再來才是畫圖，那避免我的圖還是很亂，所以又有加上所有狀態表格，以及單獨畫答案路線出來的版本

答案卷：

B1229062_林冠妤

我設定每一個 state 寫成 (x, y) ，

$x = 3L$ 水壺中目前的水量

$y = 5L$ 水壺中目前的水量

題目的意思是兩個水壺互相倒，一次倒過去目前水壺有的量，直到不能再倒過去為止，最後只要其中一個水壺是4L就達成要求(另外一個水壺中如果有水要倒掉)。

那這邊如果要直接想的話，我是想到：

1. $(0, 5)$ - 將 5L 的水壺裝滿。
2. $(3, 2)$ - 將 5L 的水壺中的3L裝過去3L的水壺(一次性裝滿)，5L 的水壺剩下 2L。
3. $(0, 2)$ - 將 3L 的水壺的水全部到掉。
4. $(2, 0)$ - 將 5L 的水壺中的2L全部倒給3L的水壺，所以5L的水壺空了。
5. $(2, 5)$ - 再將 5L 的水壺裝滿。
6. $(3, 4)$ - 再將 5L 的水壺中的1L裝給3L的水壺(因為在step 5 的時候3L的水壺就剩下 1L 就滿了)，5L 的水壺剛好剩下 4L，已找到題目要求的4L。
7. $(0, 4)$ - 最後再將 3L 的水壺中的水全部倒掉即可。

這樣就解完了

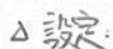
再來就是要回答問題中的Complete state diagram

1. 先設定：

代號/顏色	說明
F3	裝滿 3L 水壺
F5	裝滿 5L 水壺
E3	清空 3L 水壺
E5	清空 5L 水壺
P35	3L 水壺中的水倒過去 5L 水壺
P53	5L 水壺中的水倒過去 3L 水壺
藍色箭頭&路徑	成功量出 4L 的其中一條路徑
鉛筆箭頭&路徑	其他路徑
粉色螢光筆箭頭	self-loop

2. 然後我就開始畫全部的路徑去佐證我剛剛的想法，剛好也是畫完題目的完整狀態圖(complete state diagram(DFA))：

Team 1



1. F3 = 裝滿 3L 的水壺
2. F5 = 裝滿 5L 的水壺
3. E3 = 清空 3L 的水壺
4. E5 = 清空 5L 的水壺
5. P35 = 將 3L 水壺中的水倒至 5L 的水壺
6. P33 = 將 5L 水壺中的水倒至 3L 的水壺
7. 藍色箭頭 & 路徑 = 成功量出 4L 的一條路徑
8. 鉛筆箭頭 & 路徑 = 其它路徑

9. 粉色螢光筆箭頭 = self-loop

所有 state 的完整路徑表:

B1229062- 林冠宇

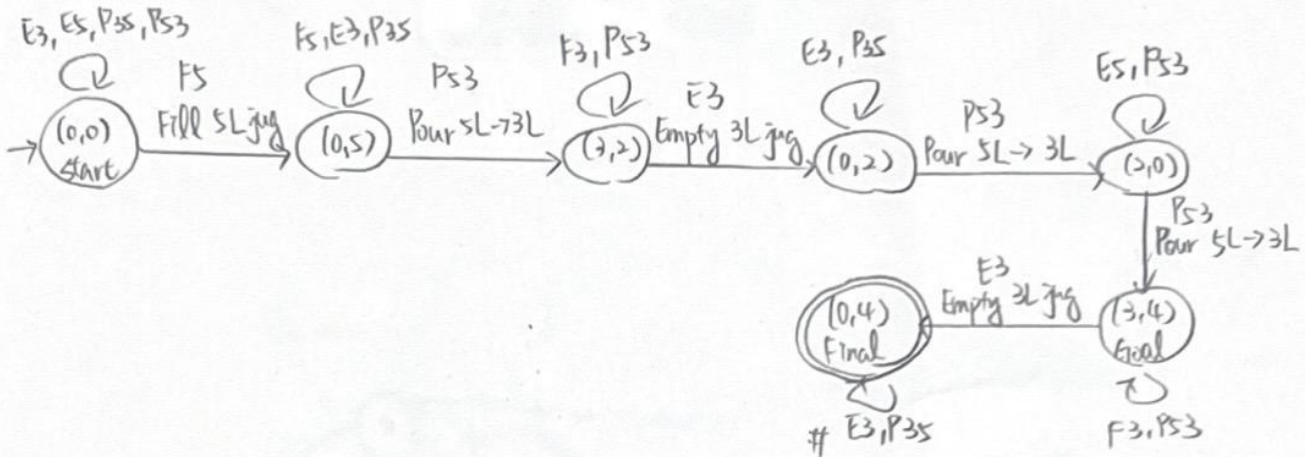
	State	F3	F5	E3	E5	P35	P53
1	(0,0)	(3,0)	(0,5)	(0,0)	(0,0)	(0,0)	(0,0)
2	(0,5)	(3,3)	(0,5)	(0,5)	(0,0)	(0,5)	(3,2)
3	(3,2)	(3,2)	(3,5)	(0,5)	(3,0)	(0,5)	(3,2)
4	(0,2)	(3,2)	(0,5)	(0,2)	(0,0)	(0,2)	(2,0)
5	(2,0)	(3,0)	(2,5)	(0,0)	(2,0)	(0,2)	(2,0)
6	(2,5)	(3,5)	(2,5)	(0,5)	(2,0)	(2,5)	(3,4)
7	(3,4)	(3,4)	(3,5)	(0,4)	(3,0)	(2,5)	(3,4)
8	(0,4)	(3,4)	(0,5)	(0,4)	(0,0)	(0,4)	(2,1)
9	(3,0)	(3,0)	(3,5)	(0,0)	(3,0)	(0,3)	(3,0)
10	(3,5)	(3,5)	(3,5)	(0,5)	(3,0)	(3,5)	(3,5)
11	(0,3)	(3,3)	(0,5)	(0,3)	(0,0)	(0,3)	(3,0)
12	(3,3)	(3,3)	(3,5)	(0,3)	(3,0)	(1,5)	(3,3)
13	(1,5)	(3,5)	(1,5)	(0,5)	(1,0)	(1,5)	(3,3)
14	(1,0)	(3,0)	(1,5)	(0,0)	(1,0)	(0,1)	(1,0)
15	(0,1)	(3,1)	(0,5)	(0,1)	(0,0)	(0,1)	(1,0)
16	(3,1)	(3,1)	(3,5)	(0,1)	(3,0)	(0,4)	(3,1)

current state → 下一個 state Next

△ 新色螢光筆 = self-loop

主要路徑 diagram:

B1229062 林冠宇



得解(也和前面 VS Code跑出的結果對的上)。

二、課程總心得：

● 心得

我每一次的作業很多都像上面這樣都會問好我的問題、我的疑惑，才會開始寫題目，那主要我也不會自己沒有吸收就交上答案，因為我也會怕 ai 給的答案不靠普，可以的話我也是會先從我的上課筆記，以及課堂 pdf 出發，也是因為我發現直接給 ai 題目要他生成的答案很多我都看不懂，然後我也要一直去查詢專有名詞。

那在學期初的幾次作業我寫的比較制式化一點，因為我也不清楚老師要得報告要多完美，也因為一開始上課那幾週我一直覺得學長姐和教授您可以有來有回的問答，如此活躍的對答我有嚇到，會覺得學長姊都好優秀，可能也是因為我是大三的關係，我就很慌張，然後上課的部分也是一直到最後一堂課，我也沒有勇氣在課堂中回答您的問題(真的很抱歉)，一方面是我又怕我講錯，二方面也是因為我大概是在期中考其一兩次上課比較跟得上，不是教授您上課的速度快，而是我在上課的過程中會有一些問題，但後面我有和同學詢問得解了，再越來越來得跟上教授速度之後，我的作業以及課程學到的也沒有什麼問題，作業、期中考也敢大方地寫下去，收穫也很多，整體蠻喜歡這堂課的。

● 再來就是期許(建議)，但也不算是，也比較算是我的心得：

- (1) 我其實一開始蠻喜歡小組討論(在教室外面的空間那裏)，因為會有自己一定要發言的過程(我可以壯膽)，然後和同學一起交流，再來就是因為一定要發言，就導致我在怎麼怕都一定要開口，雖然我很害怕，但是我蠻喜歡這個過程的；但就是後期課程，也是因為課程進度的關係吧，就沒有公開討論的部分，有點難過，但是後面教授還有小組報告討論的過程我也覺得很好，都有和同學們交流，所以我還蠻喜歡教授這堂課的教學風格。
- (2) 再來，我其實一開始會在三下選這堂課是想要在我前面每一個學期都有上教授您的課程的情況下，繼續接著上你的課程，而且也有聽到教授您之前上課有說這堂課最後會叫學生設計圖靈機，這學期選的時候教授您也有說有機會會讓我們設計，會教我們圖靈機的課程，然後我以為期末考會出要我們小組去設計圖靈機，但不是，但是後面的圖靈相關的課程我也都有好好上，學到蠻多新東西的，這次期末考我也是花了比較多時間去完成，最後畫出結果來也有種震撼感(雖然我也不知道這樣的答案可不可以)，然後我也喜歡這樣的題目，因為我也有順便去複習一下我之前學過的東西，有種程式碼、演算法、電路前後相連，都串接再一起的感覺；當然期中考的題目我也覺得很有趣，我也有額外去鑽研一下 GTKWave 怎麼用。(額外說一下我蠻喜歡圖靈的，也有看他的相關電影)

最後，從一開始我來學校後，因為電路和前一所學校學得不太一樣，而跟不上，後面有去找教授您，您都會有耐心的回答我的問題，然後有被您拉拔起來的感覺，真的很謝謝您，你的每一堂課我也都有記下所有筆記，然後我也聽得懂課程內容；最後就是謝謝教授這學期的教導，我蠻喜歡您的每一門課程，都讓我受益良多。